Capstan RL Field Guide

How the v4 quadruped learns to move, read from the code

CapstanPedal/fullbuild_v4/rl2 · 26 October 2026 · code links point at commit 231ca53; the teacher–student code (§13) is newer

The whole thing in one paragraph. A URDF made from the CAD becomes a MuJoCo simulation. 8,192 copies of the robot run at once on the GPU. Every 20 ms each robot's *policy* (a neural network) reads 153 noisy sensor numbers and outputs 12 joint targets, plus random noise so it tries something slightly different each time. Every step is scored by a weighted sum of reward terms. A second network, the *critic*, predicts how much reward is coming. After 24 steps of all 8,192 robots, PPO makes the actions that did *better than the critic expected* more likely and the rest less likely. Repeat 1,500 times (about 15–30 minutes on the RTX 4090) and the noise becomes a walk. Only the policy runs on the robot.

Contents

1 The map **2**

2 From a URDF to 8,192 robots on a GPU **2**

3 One step of one robot **3**

3.1 What the policy sees: 3 frames of 51 numbers . . . 3

3.2 Act, with noise . . . 3

3.3 Action → joint targets → torque . . . 3

3.4 Score it . . . 3

4 Did that noisy try get reward? The critic, the advantage, PPO **4**

4.1 Return and value . . . 4

4.2 Advantage: better or worse than expected? . . . 4

4.3 GAE: estimating the advantage from 24 steps . . . 4

4.4 The PPO update . . . 4

5 What happens next: four loops **5**

5.1 Choosing the next scenario . . . 5

6 Logs and saving **5**

7 Base RL: the reward you actually care about **6**

8 Reward shaping: terms that point the way **6**

8.1 Why a bell curve . . . 6

8.2 Shaping in the other tasks . . . 6

9 Reward tuning: picking the numbers **6**

10 An RL controller vs the model-based one **7**

11 Continuous commands **7**

12 How does it know trot vs walk vs crawl? **7**

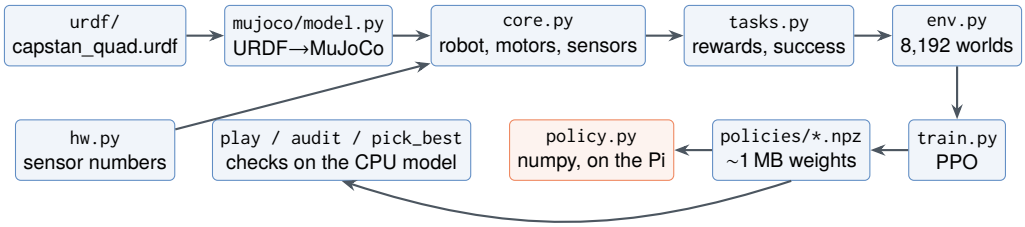
13 Privileged learning: a teacher that sees, a student that feels **8**

14 Is it good? How a policy is judged 9

15 A new build version 9

16 Words 10

1 The map



Everything for the learned controller lives in `rl2/`. It borrows the robot from two folders next to it: `urdf/` (the description generated from CAD) and `mujoco/` (which turns the URDF into a simulation and also holds the hand-written controller `gait.py`). `rl/` is the first version (walking only, ideal sensors); read `rl2`.

File	Its job
`hw.py`	The sensors as numbers: 50 Hz policy, 4–20 ms delay, gyro noise, encoder resolution, optical-flow dropouts, battery sag. Replace the assumed values with measurements.
`core.py`	Builds the model twice (light "physics" for the GPU, "full" with every mesh for checking); PD loop and motor model; `sense()`, which turns true simulator state into what the sensors would report; terrain height.
`tasks.py`	The five tasks (walk, terrain, rear, handstand, balance): what an episode is, the scenario bank, every reward term, success and fall.
`env.py`	Steps a task: `GpuEnv` runs thousands of worlds on the GPU, `CpuEnv` one world in the full model.
`train.py`	PPO: actor and critic, rollout, update, logging, held-out evaluation, snapshots. `--teacher=1`: a teacher (§13).
`distill.py`	Teacher → blind student by imitation (DAgger). No reward.
`policy.py`	The trained actor in numpy. This is what would run on the Raspberry Pi.
`play.py, audit.py, pick_best.py`	Held-out success on the full model, stress tests, choosing the best snapshot.
`terms.py, progress.py, showreel.py, compare_film.py`	Looking at runs: which reward terms dominated, films of snapshots, the all-skills film, teacher/student/blind side by side.
`run_queue.sh, experiments/`	Train a batch on cake; every batch we ran is a list file.
`check_robot.py`	Does a build's robot fit rl2? (§15)
`wandb_sync.py`	Streams the CSV logs to Weights & Biases; training itself needs no account.

2 From a URDF to 8,192 robots on a GPU

1. **Read the URDF as XML.** `mujoco/model.py` L33-L46 parses `capstan_quad.urdf` and pulls out the `<mimic>` tags: the motor rotors spin at −9× their output, and MuJoCo cannot read mimic, so they come back later as joint equalities.

```
root = ET.parse(urdf).getroot()
for j in root.findall("joint"):
    m = j.find("mimic")
    ...
spec = mujoco.MjSpec.from_string(ET.tostring(root, encoding="unicode"))
```

2. **Add what a URDF cannot say** (`mujoco/model.py` L48-L125): a free joint so the torso can fly (URDF roots are fixed), a floor and lights, 12 actuators (motor C on ab/ad, B on the hip, A on the crank), the four-bar loop closed with a *connect* equality, reflected rotor inertia (*armature*) on the driven joints, and foot sites.
3. **Make the learning version.** `core.build_spec("physics")` (`rl2/core.py` L60-L98) deletes every visual mesh and the rotor joints: the GPU only needs collision shapes, and the rotor inertia is already in the armature. The "full" version keeps everything and adds the flow and range sensor boards. Tasks add *extras*: 64 mocap boxes for the terrain course, or a ball and board.
4. **Find things by name.** `Layout` (`rl2/core.py` L115-L146) records where the 12 driven joints, four feet and the torso are in the

compiled model, and computes the standing pose from the leg IK in `gait.py`. This is why a new build works as long as the names stay.

5. **Copy it onto the GPU** (`rl2/env.py` L26-L42). `GpuEnv` compiles the model and hands it to MuJoCo Warp through MJX; one `Data` per robot, all stepped together with `jax.vmap`. Friction, mass and the terrain differ per robot, so they are passed per world.

3 One step of one robot

A *step* is one decision, 50 times per simulated second. Between decisions physics runs 10 times (500 Hz) with the joint PD loop holding the targets.

3.1 What the policy sees: 3 frames of 51 numbers

Every number passes through `sense()` (`rl2/core.py` L216-L241), so it is noisy, biased, late, quantised or missing, as on the robot.

Group	n	Source on the robot, and the sim's errors
Rotation rate (gyro) ×0.25	3	pi3hat IMU; 0.02 rad/s noise, ±0.02 rad/s bias per episode
Gravity direction	3	pi3hat attitude filter; ±1.5° tilt error per episode, 0.4° noise
Ground speed (fwd, side) ×2	2	PMW3901 optical flow × range: one step old, ±8 % scale, dropouts held, 15 % of episodes dead
Flow valid	1	the flow sensor's quality flag
Height $(h - h_0) \times 5$	1	VL53L1X time-of-flight range; 5 mm + 1 % noise
Command	3	joystick: forward, sideways (m/s), turn (rad/s); scaled ×2, ×2, ×0.25
Joint angles − standing pose	12	moteus encoders: 14-bit through 9:1, ±0.01 rad calibration offset
Joint speeds ×0.05	12	moteus velocity estimate, 0.05 rad/s noise
Last action	12	the policy's own previous output
Trot clock	2	sin, cos of a 0.3 s cycle (walking tasks; zeros otherwise)

Three frames are stacked (153 numbers), so the policy can see short trends. Each input is then normalised by a running mean and variance and clipped to ±10.

3.2 Act, with noise

The actor is an MLP, 153 → 512 → 256 → 128 → 12 with ELU activations. It outputs a mean $\mu(o)$; during training a Gaussian sample is taken with a learned, state-independent standard deviation $\sigma = e^{\texttt{log_std}}$ (initially $e^{-1} \approx 0.37$):

$$a \sim \mathcal{N}\big(\mu_\theta(o),\ \operatorname{diag}\sigma^2\big), \qquad \log \pi_\theta(a \mid o) = \sum_{j=1}^{12} \Big[-\tfrac{1}{2}\Big(\tfrac{a_j - \mu_j}{\sigma_j}\Big)^2 - \log\sigma_j - \tfrac{1}{2}\log 2\pi \Big].$$

```
mu = actor_mean(p, nm, obs)
std = jp.exp(p["log_std"])
a = mu + std * jax.random.normal(k_a, mu.shape)     # each robot gets its own noise: exploration
```

On the robot (and in every evaluation) the noise is off: the action is $\mu(o)$.

3.3 Action → joint targets → torque

Each action moves a joint target from the standing pose: 0.2 rad per unit on ab/ad, 0.6 on hip and crank, clamped to the firmware limits (`rl2/core.py` L187-L193). For 10 sub-steps the PD loop and motor model (`rl2/env.py` L51-L70) compute

$$\tau = k_p(q^* - q) - k_d\,\dot{q}, \qquad |\tau| \le \tau_{\max}\, s\, \operatorname{clip}\Big(2\big(1 - \tfrac{|\dot{q}|}{\omega_0 b}\big), 0, 1\Big),$$

with $k_p = 80/60/60$, $k_d = 1.5/2/2$ per joint, s the motor strength (0.75–1.05 per episode), b the battery sag (0.83–1.0), and the crank output A limited further by how hard the coupler can push. Full torque up to half the no-load speed, falling to zero at it. The new targets only apply after the episode's control delay (4–20 ms); before that the old ones hold.

```
tgt = jp.where(i < delay, aux["prev_target"], target)
tau = C.torque(jp, pos, vel, tgt, aux["kp"], aux["kd"], aux["strength"], aux["w0"], L.isA)
d = mjx.step(mx, d.replace(ctrl=d.ctrl.at[L.act].set(tau)))
```

3.4 Score it

The task's `end()` reads the *true* state (the simulator knows everything; the policy does not), computes every reward term, decides fallen / success / time-out, and senses the next observation (`rl2/tasks.py` L166-L194 for walk, `rl2/tasks.py` L118-L133 for the shared

`finish()`).

4 Did that noisy try get reward? The critic, the advantage, PPO

Reward arrives *every step*; there is no separate check at the end. `finish()` multiplies each raw term by its weight and adds them up, then scales by 0.1:

$$r_t = 0.1 \sum_k w_k \, x_{k,t}.$$

Success and falls are terms in this sum (+10, −10), not a separate signal: PPO never looks at "success" on its own.

4.1 Return and value

What we want to maximise is the *discounted return*, the reward from here on with later rewards counted less:

$$G_t = \sum_{l \geq 0} \gamma^l r_{t+l}, \qquad \gamma = 0.99 \;\; \text{(a reward 100 steps = 2 s ahead counts } 0.99^{100} \approx 0.37\text{)}.$$

The *critic* $V_\phi(s)$ learns to predict G_t. It may read things the robot cannot (the 22 *privileged* numbers: true velocity, true height, foot contacts, friction, added mass, motor strength, gains, delay, flow scale error, and per task 6 more; terrain now reads a 30-point height scan and foot clearances instead). The critic only exists in training, so this is allowed.

4.2 Advantage: better or worse than expected?

A balanced, standing robot always collects decent reward; that says nothing about whether its last action was good. So PPO asks whether the outcome beat the critic's prediction:

$$A_t \approx G_t - V(s_t).$$

Robot	Its noise pushed it. . .	Expected	Got	Advantage	PPO will. . .
#17	front-left foot down a bit earlier	0.80	1.00	+0.20	make that action more likely
#4,203	hip harder, body pitched	0.80	0.50	−0.30	make it less likely
#6,050	about the mean	0.80	0.81	+0.01	barely change

4.3 GAE: estimating the advantage from 24 steps

After a 24-step rollout we do not know G_t (episodes are 400 steps). *Generalised advantage estimation* mixes real rewards with the critic's guess of the rest, walking backwards (`rl2/train.py` L304-L311):

$$\delta_t = r_t + \gamma\,(1 - d_t)\,V(s_{t+1}) - V(s_t), \qquad \hat{A}_t = \delta_t + \gamma\lambda\,(1 - d_t)\,\hat{A}_{t+1}, \qquad \lambda = 0.95,$$

where $d_t = 1$ if the episode ended at t (no future after a fall). The critic's training target is $\hat{R}_t = \hat{A}_t + V(s_t)$. One subtlety: a *time-out* is not a real ending (the robot would have kept going), so on a time-out the code adds $\gamma V(s_{t+1})$ to the reward: it bootstraps from the last state instead of treating the future as zero.

```
def gae(nxt, t):
    adv_n, v_n = nxt
    nd = 1.0 - t["done"]                            # no future after a fall
    delta = t["r"] + o["gamma"] * v_n * nd - t["v"]   # reward + expected next - expected now
    adv = delta + o["gamma"] * o["lam"] * nd * adv_n
    return (adv, t["v"]), adv
```

4.4 The PPO update

For each stored action, the *ratio* $\rho_t(\theta) = \pi_\theta(a_t \mid o_t)/\pi_{\text{old}}(a_t \mid o_t)$ says how much more likely the new network makes it than the one that chose it. With advantages normalised per minibatch ($\hat{A} \leftarrow (\hat{A} - \bar{A})/\text{sd}(\hat{A})$), PPO minimises (`rl2/train.py` L283-L294)

$$L(\theta, \phi) = -\,\mathbb{E}_t\Big[\min\big(\rho_t \hat{A}_t,\; \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\Big] + c_v\,\mathbb{E}_t\big[(V_\phi(s_t) - \hat{R}_t)^2\big] - c_e\,\mathcal{H}[\pi_\theta],$$

with $\epsilon = 0.2$, $c_v = 1$, $c_e = 0.004$. The clip stops one batch from moving the policy more than ±20 % on any action; the entropy bonus $\mathcal{H} = \sum_j (\log \sigma_j + \frac{1}{2}\log 2\pi e)$ keeps some exploration. Each iteration makes 5 passes over the batch in 4 minibatches of 49,152, with Adam and gradient-norm clipping at 1.

Adaptive learning rate (rl2/train.py L365-L368): the approximate KL divergence $\mathbb{E}[(\rho - 1) - \log \rho]$ measures how far one update moved the policy. Above 2×0.01 the learning rate is divided by 1.5; below $0.01/2$ it is multiplied by 1.5 (bounds 10^{-5}–10^{-2}).

> **One try teaches almost nothing.** The policy moves because 196,608 noisy actions per update average out toward the better ones. Any single robot's experience is mostly noise.

5 What happens next: four loops

Word	How long	What it is here
Step	20 ms	One action, 10 physics steps, one reward.
Episode	≤400 steps	One try at one scenario: walk 8 s, poses 6 s. Ends early on a fall, or (terrain) on reaching the goal.
Iteration	24 steps	Every robot takes 24 steps (8,192 × 24 = 196,608 samples), then one PPO update.
Run	~1,500–2,000 iterations	One training job: 300–400 M steps, 15–30 minutes on the 4090 at 200–235 k steps/s.

The weights are fixed during the 24-step rollout and change only at the update. Episodes do *not* restart at an update: a robot halfway through a walk keeps walking with the slightly better policy. When a robot's episode ends it is reset in the same step into a new scenario. All robots run in lock-step on the GPU, so the reset is a *masked swap*: where done is true take the fresh state, else keep the old (GpuEnv.respawn).

```
new_cfg = jax.random.randint(k_c, (n,), 0, n_cfg)          # uniform; or SGS / curriculum
d2, aux2, mp, obs_n, priv_n = env.respawn(bank, d2, aux2, mp, obs2, priv2, done, new_cfg, k_r)
# inside: sel = lambda new, old: jp.where(done[:, None...], new, old)
```

A *scenario* is one row of the *bank*: everything random about an episode, fixed in advance. Commands, floor friction (0.4–1.25), added mass (−0.5 to +2 kg), motor strength, gain errors, battery sag, a shove (time and velocity), and for terrain the course itself. 1,040 scenarios for training; 4,160 different ones (another seed) for the held-out test.

5.1 Choosing the next scenario

- **Uniform**: any scenario, equally likely. What every deployable policy used.
- **SGS** (success-guided sampling, Zhang et al. 2026): each scenario keeps its last 100 outcomes; with p_i their mean,

$$w_i = \max\left((p_i + \varepsilon)^{kt}(1 - p_i + \varepsilon)^{k(1-t)},\ \varepsilon\right), \quad P(i) \propto \exp\left(\log(w_i + \varepsilon)/T\right), \quad t = 0.66,\ k = 5,\ T = 2,$$

  which peaks at scenarios succeeded about two times in three: hard but possible.
- **Level curriculum** (Rudin et al. 2021): each robot keeps a level per terrain type, up one on success, down one on failure.

What we found: over 14 runs SGS never beat uniform and lost 15–20 points on terrain (a scenario that fails early is sampled less, so it is never learned: the bank *freezes*). The curriculum gave +3 points, within seed noise; robots settled at level ~1, because up-on-success/down-on-failure settles where success is 50 %.

6 Logs and saving

When	File (on cake)	What is in it
Start	logs/<name>_opts.json	Every option and the reward weights used: a run can always be reproduced.
Every iteration	logs/<name>.csv	Steps, speed, learning rate, mean reward, success and fall rate of finished episodes, KL, critic loss, noise size, every reward term as r.<term>.
Every 10 it.	logs/<name>.out	A printed line with the four biggest reward terms.
Every 50 it.	logs/<name>_eval.jsonl	Held-out test: 4,160 unseen scenarios, mean action, success per terrain and level.
Every 50 it.	policies/snaps/<name>/<M>.npz	An actor snapshot (teacher runs also keep a .pkl).
Every 50 it., end	policies/<name>.npz, .pkl	.npz: actor + input normalisation, ~1 MB, deployable. .pkl: everything, to resume.
After the run	logs/<name>.post, .audit, .pick	run_queue.sh films it, evaluates it on the full CPU model, stress-tests it, picks the best snapshot as <name>_best.npz.

Only the deployable _best.npz files live in git; logs, checkpoints and films stay on cake. wandb_sync.py --follow streams the CSVs to Weights & Biases while a run trains.

7 Base RL: the reward you actually care about

The plainest reward is the goal itself: +10 for success, −10 for a fall, nothing otherwise. That is *sparse* (base) reward, and it is in every task as the `success` and `fall` terms. On its own it almost never works for a legged robot: a random policy falls within a second, every time, never stumbles into an 8 s walk that tracks the command, so success is never seen and the only lesson is "falling is bad". The policy learns to crouch and sit still.

8 Reward shaping: terms that point the way

Shaping adds *dense* terms that give partial credit every step, so a slightly better policy gets slightly more reward long before it ever succeeds. The whole walk reward (`rl2/tasks.py` L141-L142 weights, `rl2/tasks.py` L170-L181 raw values, `rl2/tasks.py` L111-L116 shared terms):

Term	Raw value each step	Weight	Why
`track_lin`	$\exp(-\lVert v_{xy} - c_{xy}\rVert^2/0.25)$	+1.5	The goal as partial credit
`track_yaw`	$\exp(-(\omega_z - c_\omega)^2/0.25)$	+0.75	Turning on command
`gait`	share of feet whose contact matches the trot clock	+0.5	Asks for a trot (§12)
`air`	at touch-down: air time − 0.15 s	+1.0	Real steps, not shuffling
`bounce`	v_z^2	−2.0	Smooth body
`angvel`	$\omega_x^2 + \omega_y^2$	−0.05	No rocking
`tilt`	$g_x^2 + g_y^2$ (gravity off vertical)	−5.0	Body level
`height`	$(z - z_{\text{stand}})^2$	−20	No crouch-walking
`torque`	$\sum \tau^2$	-2×10^{-4}	Energy, motor heat
`action_rate`	$\sum(a_t - a_{t-1})^2$	−0.01	No jitter: gears would suffer
`slip`	$\sum$ speed^2 of feet in contact	−0.2	Planted feet
`idle_pose`	distance from standing pose, when the command is zero	−0.2	Stand still properly
`fall`	1 on the step it falls	−10	Base reward
`success`	1 at the end of a successful episode	+10	Base reward

8.1 Why a bell curve

Tracking terms use $\exp(-e^2/\sigma)$ rather than $-e^2$: full credit at the target, smoothly less further away, never a cliff, and bounded, so one term cannot drown the rest. With $\sigma = 0.25$: error 0 → 1.00; 0.25 m/s → 0.78; 0.5 m/s → 0.37; 1 m/s → 0.02. At 0.25 m/s off, track_lin pays $0.78 \times 1.5 \times 0.1 = 0.117$ per step, about 47 over an 8 s episode, against the ±1 of success/fall after scaling: the dense terms carry most of the signal.

8.2 Shaping in the other tasks

Rear, handstand: `pose` pays for the nose pointing up (down) as a bell curve around the target angle; `feet` pays for the right pair being off the ground; `roll`, `drift` are penalties (`rl2/tasks.py` L343-L344). **Balance**: `alive` pays 1 per step not fallen, `board_level` penalises a tilted board, `feet_on` pays for feet on the board (`rl2/tasks.py` L417-L418). **Terrain**: the walk terms with gentler tilt and height penalties, so stairs and slopes are not punished (`rl2/tasks.py` L279-L280).

> **The policy optimises what you wrote, not what you meant.** The first handstand held the body at 45–50°, right at the 50° success bar, because the pose bell curve paid nearly as much there as at 64°. "Successful" 14 % of the time. That is a shaping bug.

9 Reward tuning: picking the numbers

Shaping picks the terms; tuning picks their weights (and widths such as the 0.25). The loop:

1. Train with the current weights.
2. Watch the film and read the term report (`python terms.py walk walk_try`): which term dominates, which never moved.
3. Find the term that pays for the wrong thing or does not punish it enough. Change one or two weights.
4. Retrain without editing code: `python train.py --task=walk --w=slip=-0.5,air=2.0 --name=walk_try`. An unknown term name is an error (`rl2/tasks.py` L50-L55), so typos cannot silently do nothing.

What we saw	What changed	Result
Handstand hovering at the 50° bar	pose target 0.90 → 0.87 (≈ 60°), width 0.15 → 0.05, new `margin` term (+1.0) rising smoothly from 50° to 58°	best snapshot 35 % → 82 %
Handstand resting on the body's front edge	a body corner touching the ground counts as a fall	no more leaning
Torque constraint 85 % (`--cat=0.25`)	episodes cut when a motor passed 85 %	0 %: the working handstand needs more than 85 % on 41 % of steps. Too strict a number kills the skill

Tuning is not the only lever. Walk fell 3 times in 4 when the flow sensor died. That was fixed by training with it dead in 15 % of episodes (`hw.FLOW_DEAD`): falls 75 % → 0 %. That is *domain randomisation*: changing what the robot practises rather than what it is paid for.

10 An RL controller vs the model-based one

Both send joint targets to the same PD loop and motor model; they differ in how they choose them.

`mujoco/gait.py`: written by a person.

- Picks a gait from a table: which feet are down when.
- Plans each foot: stance feet slide back at the commanded speed; swing feet land where the Raibert rule says (half a stride ahead, corrected for speed error), lifted on an arc.
- Leg inverse kinematics turns foot positions into joint angles.
- Feed-forward force $J^\top F$ so each stance leg carries its share of the weight, plus roll and pitch correction.
- Uses the true body state from the simulator.
- Predictable and explainable; does only what its author thought of. No handstand, no board on a ball; an unplanned shove or step can tip it.

`rl2`: found by trial and error.

- One network: 153 sensor numbers in, 12 joint targets out, 50 times a second.
- No foot planning, no IK, no gait table: the stepping pattern is whatever the reward favoured.
- Sees only the noisy, late readings the real robot will have.
- Trained on thousands of variations (friction, mass, weak motors, shoves, delay), so it copes with them.
- Does skills that are hard to write by hand (handstand, balance board, blind stairs). Cannot explain itself, can find loopholes in the reward, and is only as good as the simulation is honest.

11 Continuous commands

Both controllers take the same three joystick numbers: forward and sideways speed (m/s) and turn rate (rad/s). They are continuous: any mix, any value in range. A diagonal walk is two at once, e.g. forward 0.4, sideways 0.3.

In `gait.py` the command goes straight into the foot plan: stance feet slide backwards along the command vector (diagonally, for a diagonal) and the Raibert rule places swing feet ahead along it. The diagonal works because the geometry says so.

In the RL policy the command is 3 of the 51 numbers in each frame. The policy has no idea what they mean; it learns that `track_lin` pays when its body speed equals them. It can walk diagonally because it was trained on diagonals: forward $-0.5 \ldots 1.2$, sideways $-0.4 \ldots 0.4$, turn $-1.2 \ldots 1.2$, 30 % along one axis only, 12 % standing still, switching to new values 4 s into each episode (`rl2/tasks.py` L143-L158). Consequence: the policy is only good inside the ranges it saw. Ask for 2 m/s and it will do *something*, but nobody rewarded it. Ranges are a training choice, not physics.

In **terrain** a simulated operator gives the commands: speed ramps to the target, sideways and turn commands steer back to the course's centre line (`rl2/tasks.py` L328). In `teleop.py` the keyboard sets them.

12 How does it know trot vs walk vs crawl?

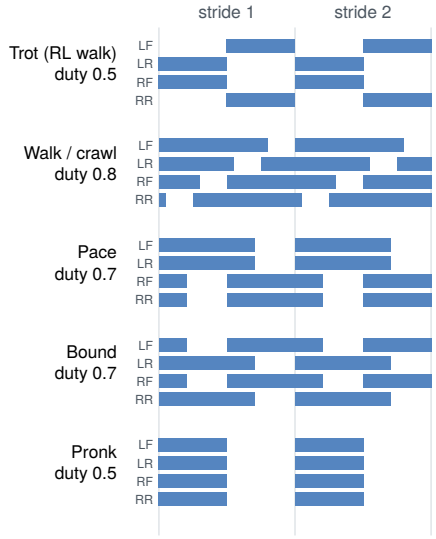
It does not, unless the reward says so. A gait is a timing pattern: which feet are down at each moment of the stride. Given only "track the speed, do not fall", RL finds whatever pattern pays most: maybe a trot, maybe a bound, a pronk, or a shuffle the simulator allows and a real robot should not do. The walk task asks for a trot two ways:

1. **A clock in the input**: the last 2 numbers of each frame are sin, cos of a 0.3 s cycle, so the policy knows where in the stride it is.
2. **A reward for following it**: `gait` pays the share of feet whose contact matches the clock. Pod i has phase offset o_i; it should be down while $(\phi + o_i) \bmod 1 < d$, with duty $d = 0.5$.

```
OFF = np.array([0.5, 0.0, 0.0, 0.5])     # per pod: LF, LR, RF, RR -> diagonal pairs: a trot
```

```
want = (phase + C.OFF) % 1.0 < 0.5       # True = this foot should be on the ground
gait = jp.mean(contact == want)
```

(The robot walks toward $-X$, so the "F" pods are its hind legs.) Other gaits are other offsets and duties; these are the ones `gait.py` uses (bars: foot on the ground, two strides):

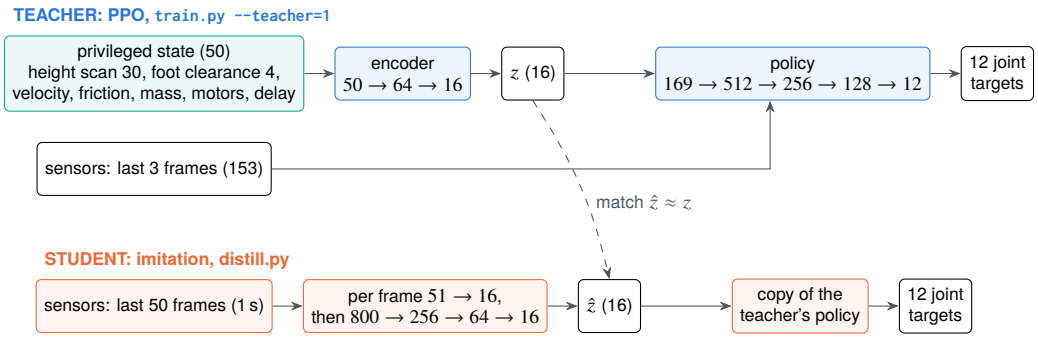


Three ways to get gaits from RL, from most control to least:

1. **Fix one gait** with a clock and a gait reward, as here. For a crawl: offsets $(0, 0.25, 0.5, 0.75)$ and duty 0.75 so three feet are always down (today the 0.5 duty is written into the `want` line, so it would become a parameter).
2. **Make the gait a command**: put offsets and duty in the observation, draw a different gait per episode; one policy learns them all and the operator picks at run time ("Walk These Ways", Margolis and Agrawal 2022).
3. **Let it emerge**: drop the gait terms, keep an energy penalty. Fu et al. (2021) found energy alone gives a walk at low speed, a trot at medium and a gallop-like gait at high, as animals do. Less control over what you get.

13 Privileged learning: a teacher that sees, a student that feels

The idea (Lee et al. 2020, "Learning quadrupedal locomotion over challenging terrain"; Kumar et al. 2021, RMA). Learning to walk blind over terrain is hard partly because the policy has to work out the ground from its legs *and* learn to act on it, at once, from a noisy reward. Split the two:



Teacher. Ordinary PPO, but its policy also reads the privileged state through a small encoder to a 16-number latent z. For terrain that state is a height scan of the course (10 points from 0.3 m behind to 1.05 m ahead × 3 across, relative to the body's height), each foot's height above the ground under it, and the hidden physics (true velocity, friction, added mass, motor strength, gains, delay). The robot has no sensor for any of it, so the teacher is not deployable: `policy.py` refuses it.

Student. Blind. It reads the last 50 frames (1 s) of the robot's real sensors. A shared per-frame layer ($51 \to 16$) compresses each frame; an MLP over the 50 compressed frames estimates $\hat{z}$; the policy is initialised as a *copy* of the teacher's and reads [last 3

frames, $\hat{z}$]. Trained by imitation, with no reward at all:

$$\mathcal{L} = \mathbb{E}_{s\sim d^{\pi_\beta}}\Big[\|\pi_S(h_s) - \mu_T(o_s, p_s)\|^2 + \|\hat{z}(h_s) - z(p_s)\|^2\Big], \qquad \pi_\beta = \begin{cases}\mu_T & \text{with prob. } \beta \\ \pi_S & \text{otherwise}\end{cases},$$

where h is the 1 s history, p the privileged state, and β falls from 1 to 0 over the first 20 % of training. This is **DAgger** (Ross et al. 2011): the states come from the *student's own* driving, so it learns what to do after its own mistakes, which plain copying of teacher trajectories never shows it.

What happened (Experiment 5, `experiments/queue5.list`, 26 Oct 2026; all held-out):

	Peak, GPU (4,160)	Best snap., CPU (200)	Last snap., CPU (200)	Stress audit (240)	Gaps (GPU)
Teacher (sees the scan)	**80.9 %**	not deployable	–	–	**52 %**
Student (blind, distilled)	78.3 %	77.0 %	**74.5 %**	**76 %**	12 %
Blind PPO (control)	70.7 %	77.0 %	55.5 %	59 %	11 %

- **Seeing is worth about 10 points, nearly all on gaps and mixed courses.** A blind robot cannot know a gap is there until a foot finds nothing; no imitation can fix that. In the films the teacher lengthens its stride before the gap; student and blind both stop at the edge.
- **The student keeps what can be felt**: stairs up 86 % vs 74 % blind; rough, slope, obstacles slightly better.
- **Best against best, blind success did not move** (77.0 % both, ±3 points at 200 episodes).
- **Distillation is stable where PPO is not.** The teacher slumped 81 → 45 %, the blind run 71 → 55 %; the student stayed in 63–78 % throughout. Supervised learning has a fixed target, which points the late-slump problem at the RL loop itself.
- **Next**: give the robot part of the scan. A forward-facing 8×8-zone range sensor (VL53L5CX) would let a student see gaps coming.

14 Is it good? How a policy is judged

- **Held-out, noise off.** Training success is measured with exploration noise on scenarios being trained on. The number that counts is success on scenarios never trained on, with the mean action.
- **The full model.** Training runs in the light GPU model; every reported score is re-measured in the full CPU model (every mesh, rotors on their joints), which the policy never saw.
- **Pick the snapshot, not the last iterate.** Held-out success swings ±15 points between evaluations 40 M steps apart and often sags late. `pick_best.py` plays the top GPU snapshots on a separate CPU *selection* bank, keeps the winner, and only then `play.py eval` gives the test number. This was the biggest single gain: terrain 50.5 → 73.5 % (3 seeds), handstand 10 → 82 %.
- **Stress audit** (`audit.py`): worst-case sensors, weak motors, slippery floor, dead flow sensor.
- **Seeds.** One run is one sample; differences under ~5 points between single runs are noise.

15 A new build version

`rl2` never names v4: it reads the robot from the folder it sits in (`core.BUILD`, or `CAPSTAN_BUILD=<path>`).

1. Build v5 as usual (CAD → meshes → `urdf/build_urdf.py`), keeping the names: `torso`, `foot_LF`…`foot_RR`, rotor joints `*_rotor*`.
2. Copy the code: `rsync -a --exclude policies --exclude logs fullbuild_v4/rl2/ fullbuild_v5/rl2/`.
3. Check it: `JAX_PLATFORMS=cpu python check_robot.py --policy=walk2_best`. Names, both models, standing on PD alone, sensors in range, and whether an old policy still walks. If that fails, retrain.
4. If the robot changed a lot, revisit the v4-tuned numbers: PD gains and action range in `core.py`, sensor values in `hw.py`, fall heights and pose targets in `tasks.py`.
5. Train on cake: `sh run_queue.sh experiments/<batch>.list`. Films come out as `video/v5_rl2_*.mp4`.

16 Words

Policy / actor	The network that picks actions; the only part that runs on the robot.
Critic / value	Predicts future reward. Training only, so it may read privileged state.
Return	Sum of future rewards, each step ahead counted 0.99× less.
Advantage	Return actually got minus what the critic expected. Positive: do that more.
GAE	Advantage estimate mixing real rewards with the critic's guess; λ trades bias for variance.
Entropy, `log_std`	How much noise the policy adds to explore. Learned; a bonus keeps it from collapsing early.
KL	How far one update moved the policy; used to adapt the learning rate.
Sparse / dense reward	Paid only at the end (success, fall) vs every step (tracking, posture).
Shaping / tuning	Choosing reward terms / choosing their weights.
Domain randomisation	Training on many versions of the world so the policy works on the real one.
Scenario bank	Fixed list of episode settings; training draws from one, testing uses another.
Privileged information	What the simulator knows and the robot cannot sense (terrain ahead, friction).
Teacher–student, DAgger	Train with privileged info, then imitate it from real sensors, on the student's own states.
Held-out evaluation	Testing on scenarios never trained on, without noise. The only fair score.
Sim-to-real gap	Everything the simulation gets wrong about the robot; `hw.py` exists to shrink it.

Results, films and the whole run history: `rl2/sgs-lessons.md`; the results pages on claude.ai; the `capstan-rl2` project on Weights & Biases.